



12.3, 12.4 Transformers

안재국 / 2026.04.09

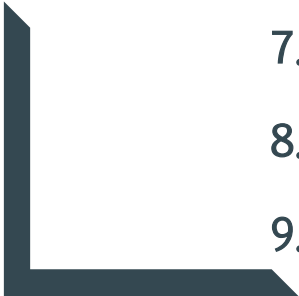


Computational Data Science LAB



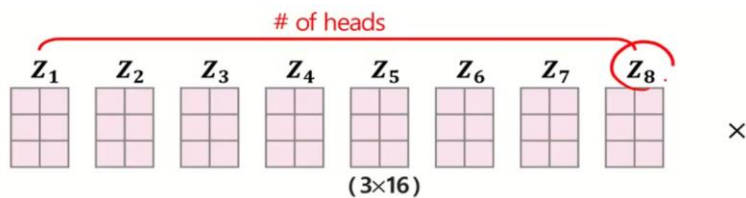
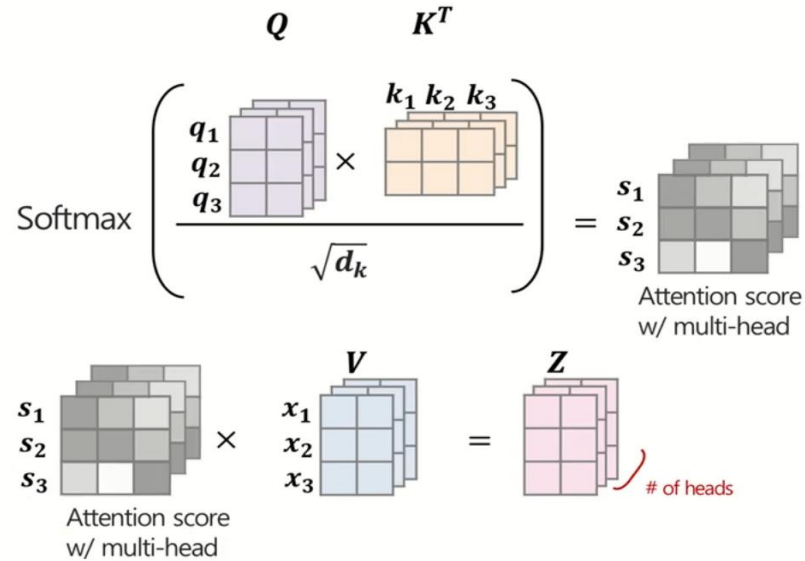
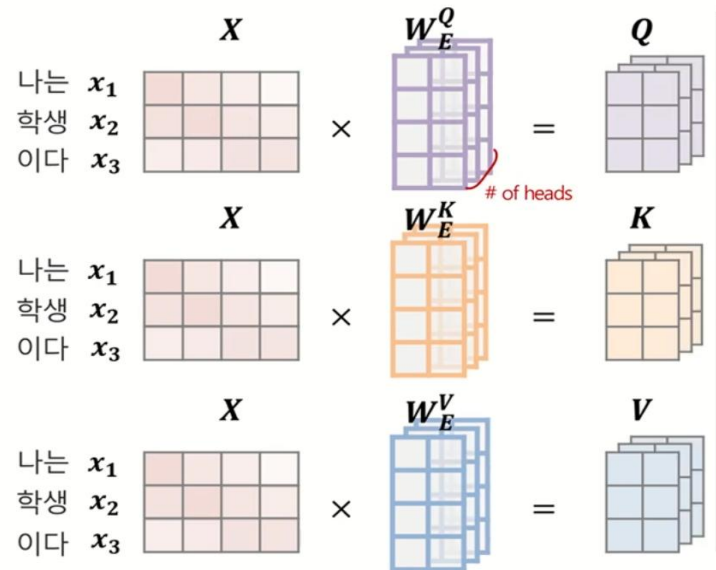
CONTENTS



1. Remind
 2. Decoder Transformers
 3. Masked Self-Attention
 4. Self-Supervised Training
 5. Encoder Transformers
 6. Sequence-to-Sequence Transformers
 7. Vision Transformers
 8. Audio Data
 9. Text-to-Speech
- 
- 

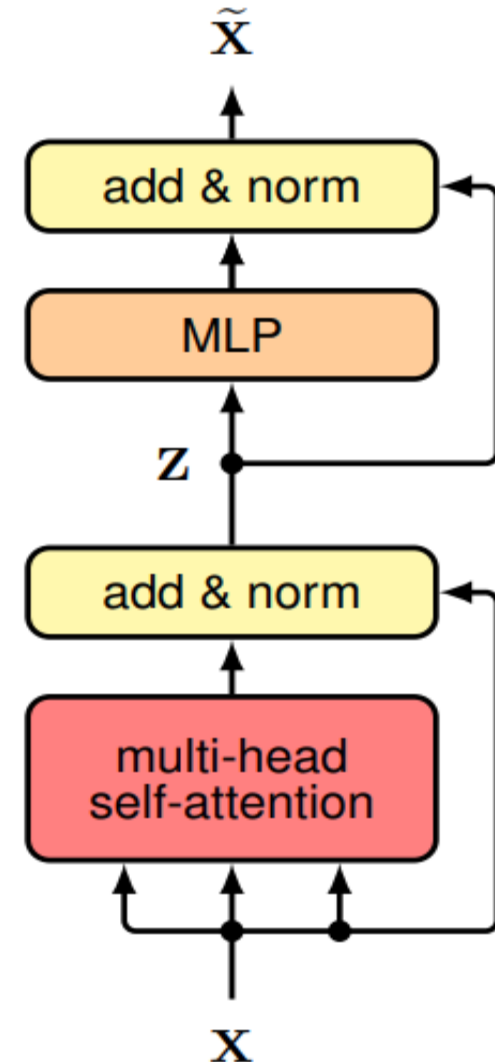
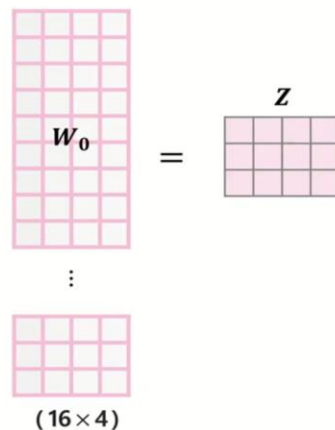
00 | Transformers

Remind



"Transformer is designed to preserve the spatial and temporal structure"

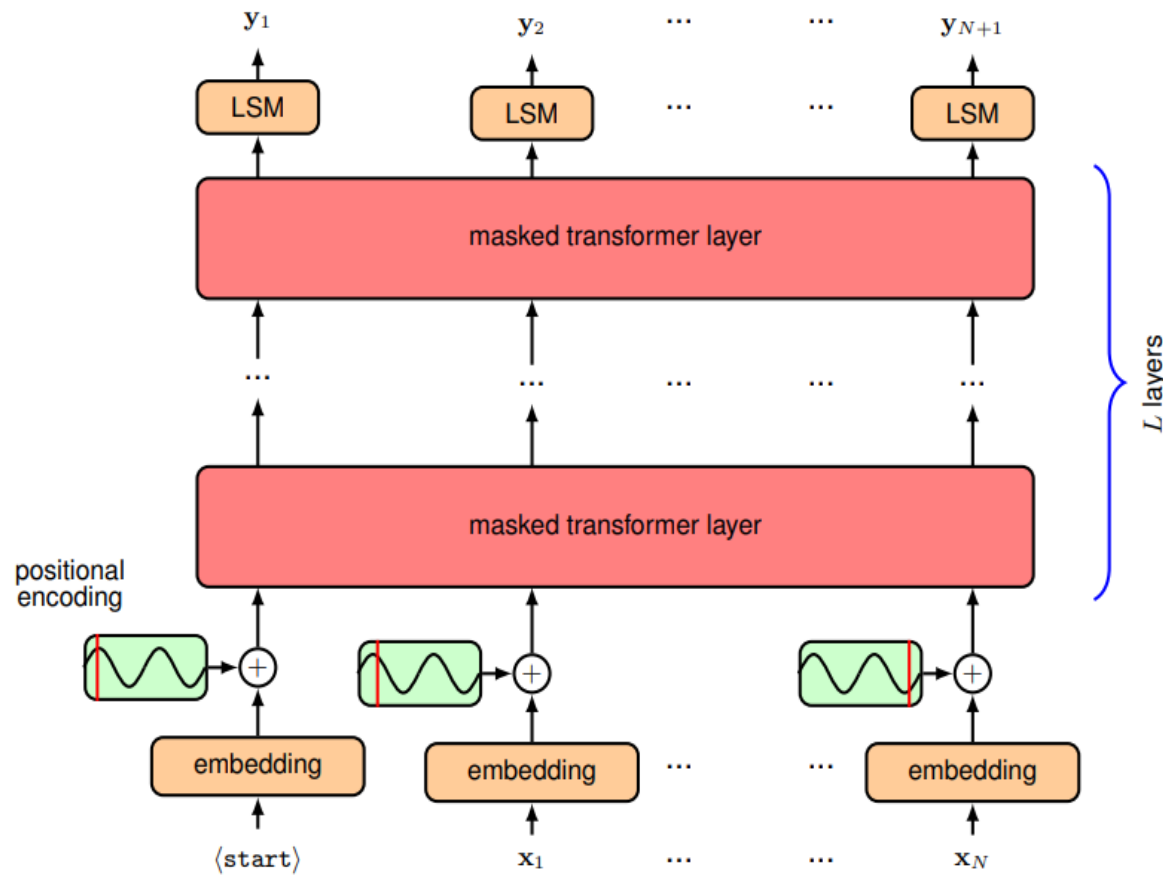
Embedded vector Z 가 입력 데이터를 모든 차원/시점을 고려하여 차원축소 하는 과정에서 입력데이터와 차원은 동일하게 유지



01 | Decoder Transformers

Autoregressive Sequence Generation

- Idea
 - ✓ decoder-only transformer는 출력 시퀀스를 생성하는 generative model
 - ✓ 대표적인 예가 GPT (Generative Pretrained Transformer)
 - ✓ 이전 토큰들이 주어졌을 때 다음 토큰의 확률분포를 예측
- Autoregressive view
 - ✓ $\langle \text{start} \rangle \rightarrow \text{I}$, $\langle \text{start} \rangle, \text{I} \rightarrow \text{swam}$
 - ✓ 새로 생성된 토큰은 다시 입력에 포함되어 다음 토큰 생성에 사용
- Output layer
 - ✓ transformer의 각 출력 벡터는 vocabulary 차원으로 선형변환됨
 - ✓ 최종 벡터는 “swim류 단어가 어울린다”와 같은 문맥 정보를 담은 벡터이므로, 이를 단어장 크기로 선형변환하여 각 단어 후보의 점수를 계산
 - ✓ 이후 softmax를 적용해 다음 토큰의 확률분포를 얻음
 - ✓ $Y = \text{Softmax}(\tilde{X}W^{(p)})$ swam: 0.59, swim: 0.24



02 | Masked Self-Attention

Preventing Look-Ahead

- Why masking?

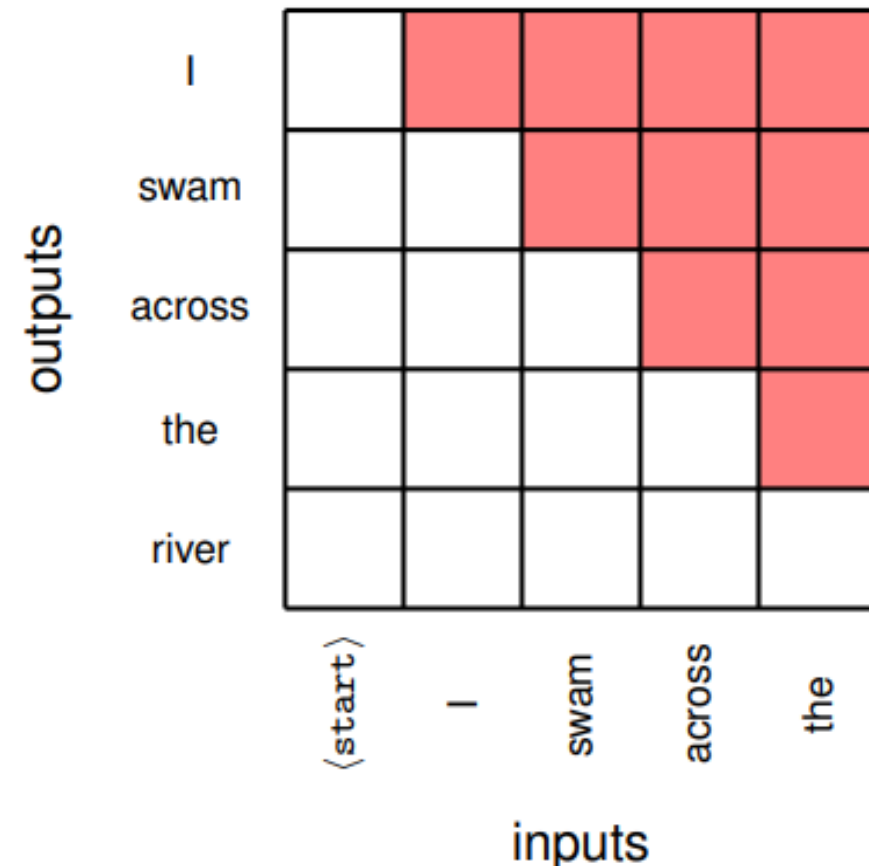
- ✓ 문장 전체를 한 번에 넣고 학습하면 모델이 미래 토큰을 미리 볼 수 있는 문제가 생김
- ✓ 이 경우 정답을 보고 복사하는 식으로 학습할 수 있음

- Causal attention

- ✓ self-attention에서 현재 위치보다 뒤의 토큰에 대한 attention weight를 0으로 설정
- ✓ 이 방식을 masked attention 또는 causal attention 이라고 함
- ✓ 결과적으로 현재 토큰 예측은 과거 토큰들에만 의존

- Parallel processing

- ✓ GPT는 병렬 계산을 하면서도 미래 정보 누설 없이 next-token prediction을 수행
- ✓ 학습에서는 정답 문장 <start> I swam across the가 이미 주어지므로, <start>→I, I→swam, swam→across, 를 한 번에 병렬 계산
- ✓ 반면 추론에서는 처음에 <start>만 있으므로 먼저 I를 생성하고, 그다음 <start> I로 swam, 다시 <start> I swam으로 across를 예측해야 해서 한 토큰씩 순차적으로 생성



03 | Self-Supervised Training

Next-Token Prediction

- Training data
 - ✓ GPT는 **unlabelled natural language corpus** 로 학습 가능
 - ✓ 각 training sample은 입력 시퀀스와 **다음 토큰 target** 으로 구성됨
 - ✓ 즉, 문장 자체가 supervision 역할을 수행
- Example
 - ✓ I swam across → target: the
 - ✓ I swam across the → target: river
 - ✓ 하나의 문장에서 여러 개의 next-token training pair를 만들 수 있음
- Inference
 - ✓ 학습된 모델은 다음 토큰의 확률분포를 출력
 - ✓ I swam across 출력 확률 예: the 0.72, a 0.08, river 0.03
 - ✓ 선택된 토큰을 다시 입력에 붙여 **반복적으로 문장 생성**
- Efficiency
 - ✓ 전체 시퀀스를 병렬 처리하여 학습 효율을 높임

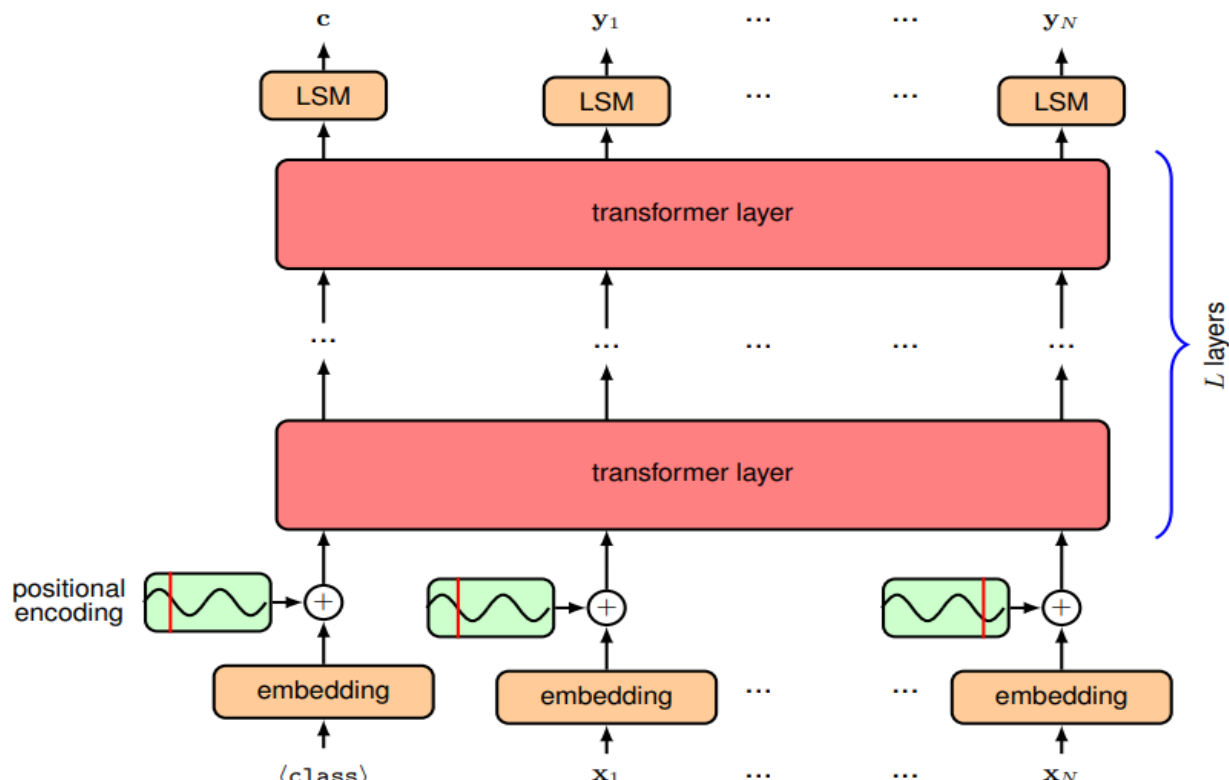
I swam across the river to get to the other bank.

04 | Encoder Transformers

Bidirectional Context Understanding

- Idea
 - ✓ encoder transformer의 대표 예는 BERT
 - ✓ 입력 시퀀스를 받아 **문맥 표현**을 학습하는 모델
 - ✓ decoder와 달리 **출력 시퀀스를 생성하지 않고**, 입력 이해에 초점
- Bidirectional property
 - ✓ 특정 토큰을 예측할 때 **앞 단어와 뒤 단어를 모두 참고 가능**
 - ✓ decoder의 look-ahead masking가 필요 없음
- Pre-training
 - ✓ 입력 토큰 일부를 **<mask>** 로 바꾸고 원래 단어를 예측
 - ✓ 전체 토큰 중 약 **15%** 를 예측 대상으로 사용
 - ✓ 예: I <mask> across the river to get to the <mask> bank
 - ✓ 문장을 이해하는 내부 표현을 잘 만들도록
가중치 파라미터들을 훈련

- Fine-tuning
 - ✓ 문장 분류(감정 분석 등)는 c만 사용, token-level task(NER 등)는 $y_1 \sim y_n$ 만 사용



05 | Sequence-to-Sequence Transformers

Encoder-Conditioned Sequence Generation

- Idea

- ✓ Encoder와 Decoder를 결합한 구조
- ✓ 예: 영어 문장을 입력으로 받아 한국어 문장을 생성하는 번역 문제
- ✓ 입력 시퀀스를 읽고, 그 정보를 바탕으로 출력 시퀀스를 한 토큰씩 생성

- Encoder representation

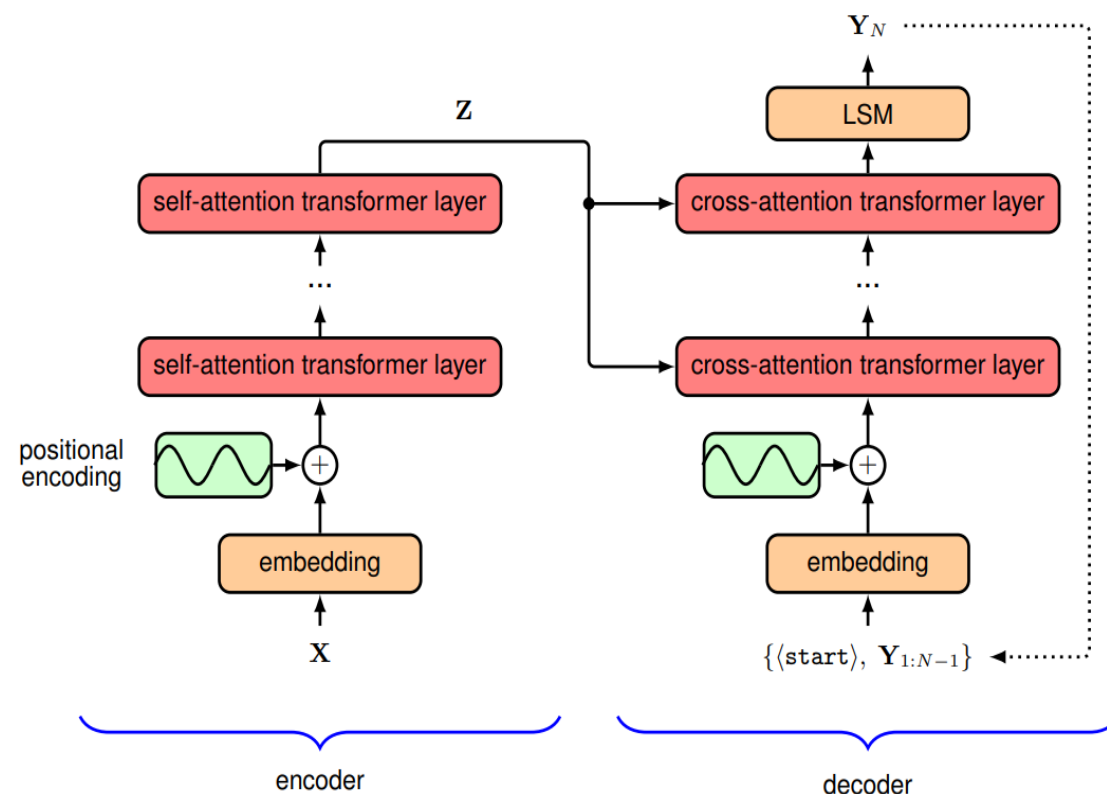
- ✓ Encoder는 입력 문장 영어 I am a student를 읽어서 각 위치별로 문맥이 반영된 표현들 Z 를 만들
- ✓ Z 는 각 토큰이 문장 전체 문맥을 반영한 벡터 (양방향이므로 앞뒤 모두 참조)

- Cross-attention

- ✓ Decoder는 자기 자신이 지금까지 생성한 토큰들로부터 현재 필요한 정보를 먼저 계산
- ✓ query는 decoder의 masked self-attention 출력에서 생성 → "지금 뭐가 필요한지" 파악 (마찬가지로 Z 에 w_Q^{cross} 를 곱해서 생성)
- ✓ key와 value는 encoder 출력 Z 에 w_k^{cross} , w_v^{cross} 를 곱해서 생성 → "원문의 어떤 정보가 있는지" 제공

- Difference

- ✓ GPT (Decoder only): 이전에 생성한 토큰들만 보고 다음 토큰 예측
- ✓ Encoder-Decoder: Z 를 함께 참고하여 다음 토큰 예측

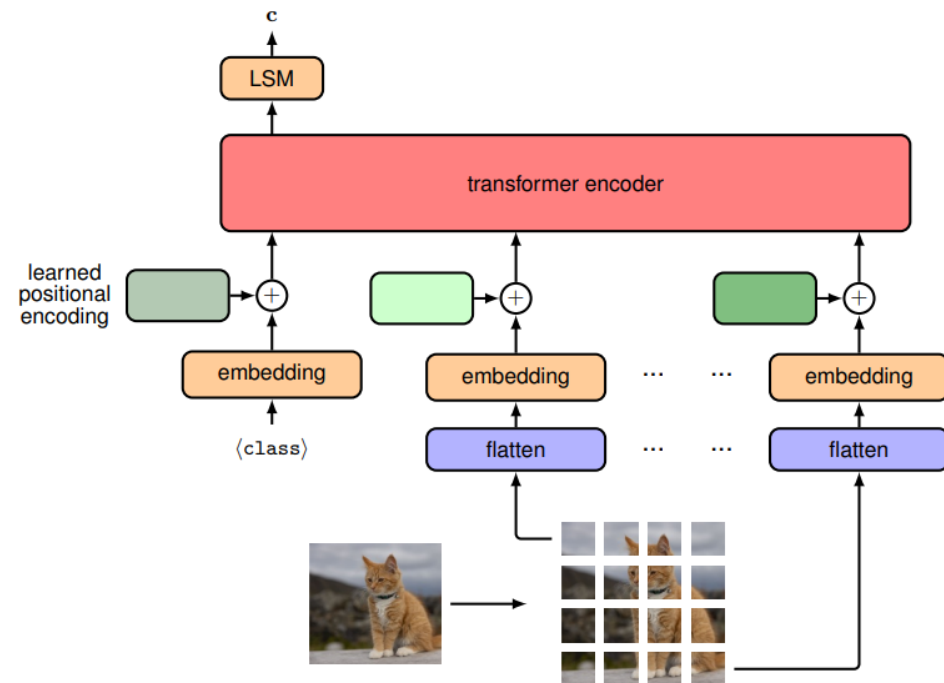


06 | Vision Transformers

Applying Transformer Encoders to Images

- Idea
 - ✓ vision transformer는 transformer encoder를 이미지에 적용한 구조
 - ✓ 입력 이미지를 토큰 형태로 바꾼 뒤 encoder에 넣음
- Tokenization
 - ✓ 이미지를 픽셀 단위가 아니라 patch 단위로 분할
 - ✓ 각 patch를 flatten 하여 하나의 토큰 벡터로 변환
 - ✓ 예를 들어 64×64 이미지에 patch 크기가 16×16 이면
가로: $64/16 = 4$ 세로: $64/16 = 4$ 총 patch 수는 $4 \times 4 = 16$
flatten 시키면 $16 \cdot 16 \cdot 3 = 768$ 차원 벡터
- Input Structure
 - ✓ 입력 앞에는 learnable <class> token 을 추가
 - ✓ 각 patch embedding에 learned positional encoding 을 더함
 - ✓ encoder 출력 중 <class> token의 출력을 이용해 최종 class label 예측

- Comparison with CNN
 - ✓ ViT는 CNN보다 이미지 구조를 미리 가정하지 않음
 - ✓ 따라서 더 많은 데이터가 필요한 경우가 많지만 충분한 데이터가 있으면 더 높은 성능에 도달 가능
 - ✓ 가정이 없다는 건 반대로 제약도 없다는 뜻
CNN은 locality 가정 때문에 오히려 놓치는 패턴이 있을수도 있음



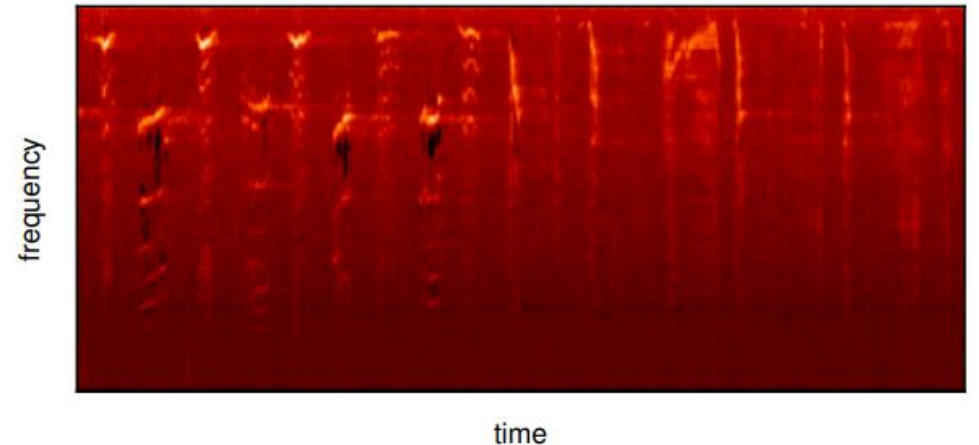
07 | Audio Data

Applying Transformers to Audio Classification

- Audio representation
 - ✓ 오디오 신호는 보통 waveform으로 저장되지만, 실제로는 mel spectrogram 으로 전처리하는 것이 효과적
 - ✓ mel spectrogram은 시간(time) × 주파수(frequency) 형태의 행렬
- Transformer approach
 - ✓ mel spectrogram을 patch 단위로 분할 하여 token화
 - ✓ 각 patch를 flatten한 뒤 positional encoding을 더해 transformer encoder 에 입력
 - ✓ 마지막 <class> token 출력으로 오디오 class 예측
- Why transformer?
 - ✓ CNN은 local relation에는 강하지만 긴 거리 의존성 파악에는 한계가 있음
 - ✓ transformer는 더 긴 문맥 정보를 반영할 수 있어 audio classification에 유리

- Key point

- ✓ audio data에서도 핵심은 mel spectrogram을 token sequence처럼 처리하여 transformer encoder로 분류하는 것



08 | Text-to-Speech

Conditional Language Modelling for Speech Synthesis

- Idea

- ✓ text-to-speech는 주어진 텍스트를 음성으로 생성 하는 문제
- ✓ 단순 회귀보다, 이를 language modelling 문제로 보는 것이 더 적절

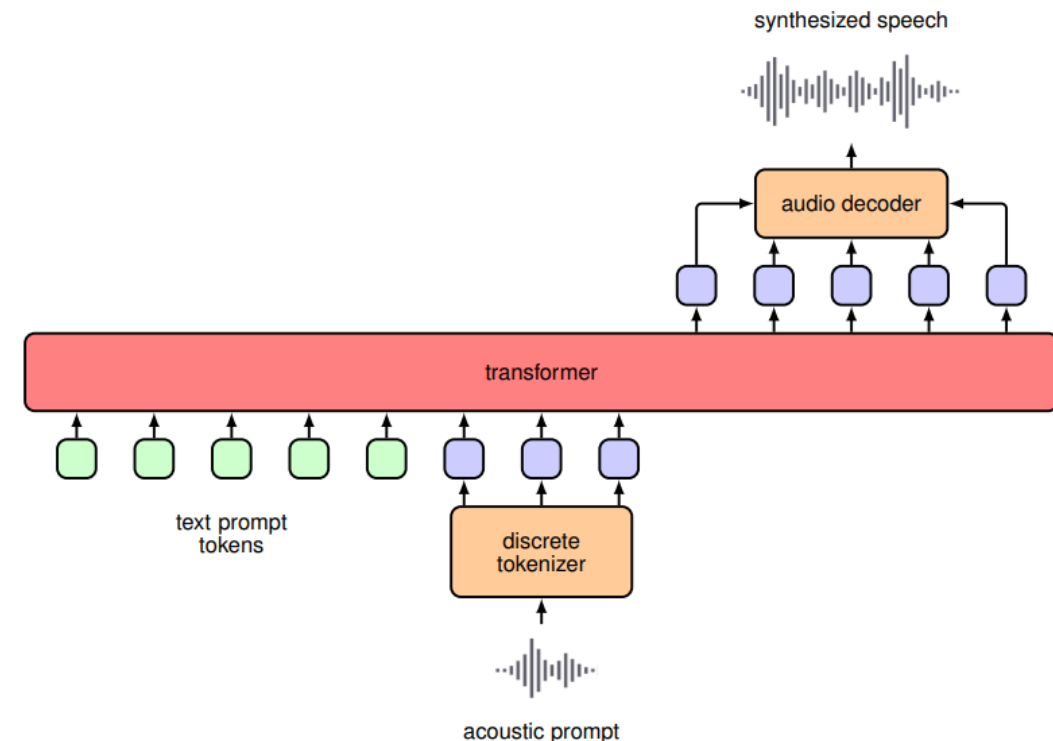
- Tokenization and conditioning

- ✓ Step 1: 텍스트 토큰화 "안녕하세요" → 서브워드 토크나이저 → [523, 89, 412]
- ✓ Step 2: 화자 음성 토큰화 화자 3초 샘플 음성 → discrete tokenizer (encoder(EnCodec) + codebook quantization) → acoustic prompt token ID [42,178, ...,901]
- ✓ Step 3: Embedding text token IDs와 acoustic prompt token IDs를 각각 embedding하여 transformer 입력용 D차원 벡터로 변환
- ✓ Step 4: 입력 구성 D차원 벡터들을 이어붙임 + positional encoding
- ✓ Step 5: Autoregressive 생성 vector → $\text{Linear}(D \rightarrow |V_{\text{speech}}|) \rightarrow \text{softmax}$ → 생성된 토큰을 뒤에 붙이고 반복(ex) speech token 12가 생성되면 다시 인풋에 넣어 다음 토큰 생성)

- ✓ Step 6 : Codebook lookup

생성된 speech token IDs → codebook에서 해당 acoustic vectors 조회

- ✓ Step 7: Audio Decoder codebook vector sequence → audio decoder → 화자 A 스타일의 “안녕하세요” 음성 합성



Q&A

감사합니다.